

วันที่ 2

Arrays, Sets และ Enum Types

สร้างโมเดลที่ยืดหยุ่น — รองรับข้อมูลทุกขนาด



Arrays

จัดการตัวแปรหลายตัว



Sets

ชุดค่าและการ filter



Enum Types

ชื่อแทนตัวเลข

กำหนดการวันที่ 2

09:00–10:00

**Arrays และ Sets***การประกาศ, การเข้าถึง, set operations*

10:00–11:00

**forall, sum, Comprehensions***aggregation functions, generators, where clause*

11:00–12:00

**Workshop 2A***Generic Production Planning — ทดลองทำเอง*

13:00–14:00

**Enumerated Types***enum declaration, array indexing, data file*

14:00–15:00

**Conditional Expressions***if-then-else, bool2int, forall+where*

15:00–16:00

**Workshop 2B***Multi-Product Multi-Line — allowed lines, minimize cost*

ทำไมต้องใช้ Arrays?

ปัญหาจริง: จำนวนตัวแปรขึ้นอยู่กับข้อมูล — ไม่รู้ล่วงหน้าว่ามีกี่สินค้า ก็เครื่องจักร

❌ ไม่ใช่ Arrays (แย)

```
var 0..100: p1;  
var 0..100: p2;  
var 0..100: p3; % ...  
  
constraint 3*p1+5*p2+2*p3<=120  
  
% ถ้าเพิ่มสินค้า → แก้ทุกบรรทัด!
```

✅ ใช้ Arrays (ดี)

```
int: n = 3;  
array[1..n] of int: labor;  
array[1..n] of var 0..100: p;  
  
constraint  
  sum(i in 1..n)  
    (labor[i]*p[i]) <= 120;
```



เพิ่มสินค้าได้เพียงแก้ n และ data — โมเดลไม่ต้องแก้!

การประกาศ Arrays

1D Parameter Array

```
array[1..5] of int: profit = [400,300,500,200,450];
```

1D Decision Variable Array

```
array[1..n] of var 0..100: produce;
```

2D Array (ตาราง)

```
array[JOB, MACHINES] of int: time_needed;  
% เข้าถึงด้วย: time_needed[j, m]
```

Array Literal (inline)

```
array[1..3,1..2] of int: mat =  
  [| 1, 2 | 3, 4 | 5, 6 |];  
% | คั่นระหว่างแถว
```

Sets — ชุดของค่า

```
% Set declarations
set of int: MACHINES = {1,2,3};
set of int: DAYS = 1..7;           % range
set of int: WEEKEND = {6, 7};
```

```
% ตรวจสอบ membership
constraint day in DAYS;
```

ประยุกต์ใช้จริง:

```
% กรองด้วย where: ทำงานเฉพาะวันทำงาน
set of int: WORKDAYS = 1..5;
constraint forall(d in DAYS where d in WORKDAYS)(
  workers[d] >= min_workers % เฉพาะวันธรรมดา
);
```

Set Operations:

`in` สมาชิก: 3 in {1,2,3}

`union` รวมเซต: A union B

`intersect` ตัดกัน: A intersect B

`diff` ผลต่าง: A diff B

`card(S)` จำนวนสมาชิก

`subset` เป็นเซตย่อย

forall และ sum — หัวใจของ MiniZinc

```
forall( generator )( constraint )
```

```
% ทุก resource ต้องไม่เกิน capacity
```

```
constraint forall(r in RESOURCES)(  
    sum(p in PRODUCTS)  
        (consumption[p,r] * produce[p]) <= capacity[r]  
);
```

```
sum( generator )( expression )
```

```
% รวมกำไรทุกสินค้า
```

```
var int: total_profit =  
    sum(p in PRODUCTS)(profit[p] * produce[p]);  
solve maximize total_profit;
```

```
where — filter generator
```

```
% กรองเฉพาะที่เกี่ยวข้อง (หลีกเลี่ยง div by 0)
```

```
forall(r in RES)(sum(p in PRD where c[p,r]>0)  
    (c[p,r]*produce[p]) <= cap[r]);
```

List Comprehensions

```
[ expression | generator where condition ]
```

```
% สร้าง list ของกำไรรวมแต่ละสินค้า  
[profit[p]*produce[p] | p in PRODUCTS]  
  
% Output: แสดงเฉพาะสินค้าที่ผลิต (> 0)  
[ show(p) ++ " = " ++ show(produce[p]) ++ "\n"  
  | p in PRODUCTS where fix(produce[p]) > 0 ]  
  
% นับจำนวนวันที่ OT (bool2int)  
sum(d in DAYS)(bool2int(overtime[d] > 0))  
  
% exists: มีอย่างน้อยหนึ่งที่เป็นจริง  
constraint exists(p in PRODUCTS)(produce[p] > 5);
```

ตัวอย่างสมบูรณ์: Multi-Product Planning

```
int: n = 4;    % สินค้า 4 ชนิด
array[1..n] of int: profit =
    [400, 300, 500, 200];
array[1..n] of int: weight =
    [3, 2, 5, 1];
int: capacity = 10;

array[1..n] of var 0..5: produce;

constraint
    sum(i in 1..n)
        (weight[i]*produce[i]) <= capacity;

solve maximize
    sum(i in 1..n)(profit[i]*produce[i]);
```

◀ กำหนด n ครั้งเดียว
ตัวแปรอื่นตามอัตโนมัติ

◀ 1 constraint ครอบ
ทุก resource

◀ objective ก็ใช้ sum
ไม่ต้อง hardcode



Workshop 2A — Generic Production Planning

เวลา: 60 นาที | ไฟล์: workshop2a_starter.mzn + workshop2a_data.dzn

1

เพิ่ม Constraint

เติม forall(r in RESOURCES)(...) แทน TODO

2

เพิ่ม solve maximize

เติม solve maximize ... ;

3

รันกับ data file 1

minizinc workshop2a_starter.mzn workshop2a_data.dzn — ตรวจสอบผล

4

เปลี่ยน data file

เปลี่ยนเป็น workshop2a_data2.dzn (6 สินค้า) — โมเดลเดิมรันได้เลย!

5

(ท้าทาย) Min requirement

เพิ่ม constraint: สินค้าแรกต้องผลิตอย่างน้อย 3 หน่วย

ช่วงบ่าย

Enumerated Types และ Conditional Expressions

- enum declarations

- array indexing ด้วย enum

- if-then-else-endif

- bool2int()

Enumerated Types — ชื่อแทนตัวเลข

แทนที่จะใช้ 1,2,3 — ใช้ชื่อจริง: ChairA, ChairB, TableX ทำให้อ่านและ debug ง่ายขึ้นมาก

❌ ก่อน — ใช้ตัวเลข

```
% หมายเลข 1=Gear, 2=Axle, 3=Frame
array[1..3] of int: profit;

% output แสดง 1, 2, 3 — ไม่รู้คืออะไร

constraint produce[1] != produce[2];

% produce[1] คืออะไร?
```

✅ หลัง — ใช้ Enum

```
enum PRODUCTS = { Gear, Axle, Frame };
array[PRODUCTS] of int: profit;

% output แสดง Gear, Axle, Frame

profit[Gear] % ชัดเจน!

profit[Axle] % ไม่สับสน
```

การประกาศและใช้งาน Enum Types

```
% — กำหนดใน model —
enum MACHINES = { LaserCutter, Welder, Assembler };
enum PRODUCTS = { ChairA, ChairB, TableX };

% — กำหนดใน data file (.dzn) —
enum PRODUCTS; % ประกาศใน .mzn ไม่กำหนดค่า
% ใน .dzn: PRODUCTS = { Bolt, Nut, Bracket };

% — ใช้งาน —
array[PRODUCTS] of int: profit; % index ด้วย enum
constraint forall(m in MACHINES)(load[m] <= cap[m]);
```

จุดสำคัญ:

Enum ใน model vs. data file	ถ้าประกาศใน .mzn ไม่ใส่ค่า → กำหนดค่าใน .dzn ได้
ใช้เป็น array index	array[MACHINES] of int — ใช้ชื่อเครื่องแทนตัวเลข
Enum ใน output	show(p) จะแสดง 'ChairA' ไม่ใช่ '1'

Generic Model — Enum + Data File

 production.mzn

```
enum PRODUCTS;
enum RESOURCES;

array[PRODUCTS] of int: profit;
array[RESOURCES] of int: cap;
array[PRODUCTS,RESOURCES]
  of int: consume;

array[PRODUCTS]
  of var 0..100: produce;

constraint forall(r in RESOURCES)(
  sum(p in PRODUCTS)
    (consume[p,r]*produce[p])<=cap[r]);

solve maximize sum(p in PRODUCTS)
  (profit[p]*produce[p]);
```

 factory_a.dzn

```
PRODUCTS = { A, B };
RESOURCES = { Labor, Mat };

profit = [800, 1200];
cap = [120, 150];
consume = [| 3, 5 | 5, 2 |];

% เปลี่ยนโรงงาน → แก้ .dzn เท่านั้น:

% PRODUCTS = { P1,P2,P3,P4 };

% profit = [600,900,750,1100];

% ...
```

 Model เดิม + Data ต่างกัน = แก้ปัญหาต่างโรงงานได้ทันที!

Conditional Expressions

```
if <bool> then <expr1> else <expr2> endif
```

ใน Constraints

```
% สินค้าที่ไลน์อนุญาตเท่านั้นที่ผลิตได้
constraint forall(p in PRODUCTS, l in LINES)(
  if l in allowed[p] then true else produce[p,l]=0 endif
);
```

ใน Output

```
% แสดงสถานะแตกต่างกัน
(if fix(ot[d]) > 0 then "OT"
 else "normal" endif)
```

bool2int(expr) = 1 ถ้าเป็นจริง, 0 ถ้าเป็นเท็จ → $\text{sum}(d \text{ in } \text{DAYS})(\text{bool2int}(\text{ot}[d]>0))$ = นับวัน OT

ตัวอย่าง: Production with Overtime

```
int: n = 6;    % 6 วัน
array[1..n] of int: demand = [10,15,8,20,12,5];
int: regular_cap = 12;    % ผลิตปกติสูงสุด

array[1..n] of var 0..20: reg;    % ปกติ
array[1..n] of var 0..20: ot;    % OT

constraint forall(d in 1..n)(
  reg[d] + ot[d] >= demand[d] /\ reg[d] <= regular_cap);

var int: cost =
  sum(d in 1..n)(100*reg[d] + 150*ot[d]);
var int: ot_days = sum(d in 1..n)(bool2int(ot[d]>0));

solve minimize cost;
```

`bool2int(ot[d] > 0) = 1` ถ้าวันนั้นมี OT, 0 ถ้าไม่มี — `sum` นับจำนวนวันที่ใช้ OT ได้เลย!

forall + where — กรองเงื่อนไข

```
% where กรองเฉพาะสิ่งที่ต้องการ

% 1. เฉพาะ resource ที่สินค้านั้นใช้จริง (หลีกเลี่ยง div by zero)
int: max_p = max(p in PRODUCTS)(
  min(r in RESOURCES where consume[p,r] > 0)(cap[r] div consume[p,r]));

% 2. เฉพาะสินค้าที่ได้รับอนุญาตสำหรับไลน์นั้น
constraint forall(p in PRODUCTS, l in LINES)(
  if not (l in allowed[p]) then produce[p,l] = 0 endif);

% 3. Output: แสดงเฉพาะ row ที่มีค่า
[ show(p)++ " บน " ++ show(l) ++ ": " ++ show(produce[p,l]) ++ "\n"
  | p in PRODUCTS, l in LINES where fix(produce[p,l]) > 0 ]
```

where clause ใช้ได้ใน: forall, sum, exists, array comprehension — เป็น pattern ที่ใช้บ่อยมาก



Workshop 2B — Multi-Product Multi-Line

เวลา: 60 นาที | ไฟล์: workshop2b_starter.mzn + workshop2b_data.dzn

โจทย์: โรงงานมีไลน์ผลิต 3 ไลน์ และสินค้า 5 ชนิด
แต่ละสินค้าใช้ได้แค่ไลน์ที่กำหนด (set of LINES: allowed)
ต้นทุนต่อหน่วยต่างกันในแต่ละไลน์ — minimize ต้นทุนรวม

- 1 เพิ่ม constraint: ไลน์ที่ไม่อนุญาต → produce = 0:
- 2 เพิ่ม constraint: แต่ละไลน์ไม่เกิน line_capacity:
- 3 เปลี่ยน solve satisfy → solve minimize total_cost:
- 4 ตรวจสอบผล: แต่ละสินค้าใช้ไลน์ที่ถูกที่สุดหรือเปล่า? ทำไม?
- 5 (ท้าทาย) ห้ามใช้ Line2+Line3 พร้อมกัน:

Pattern ที่ใช้บ่อยในโมเดลจริง



Resource Capacity

```
constraint forall(r in R)(sum(p in P)(c[p,r]*x[p]) <= cap[r]);
```

ทุก resource ต้องไม่เกิน — ใช้ใน scheduling, production, routing



Enum + Data File

```
enum PRODUCTS; array[PRODUCTS] of int: profit;
```

โมเดลยืดหยุ่น รองรับข้อมูลทุกขนาด ไม่ต้องแก้ model



Conditional Assignment

```
if l in allowed[p] then true else produce[p,l]=0 endif
```

กำหนดเงื่อนไขซับซ้อน เช่น เส้นทางอนุญาต, กะทำได้



Count with bool2int

```
sum(d in DAYS)(bool2int(ot[d] > 0))
```

นับจำนวนกรณีที่เกิดขึ้น เช่น วัน OT, เครื่องที่ใช้

Quick Reference: Arrays

```
% — DECLARATION —
array[1..n] of int: name;           % 1D parameter
array[1..n] of var 0..100: name;    % 1D variable
array[ENUM1, ENUM2] of int: name;   % 2D enum-indexed
```

```
% — ACCESS —
name[i]           % 1D access
name[i, j]        % 2D access
name[Enum1]        % enum-indexed access
```

```
% — AGGREGATION —
```

```
sum(i in 1..n)(a[i])           % รวม
min(i in 1..n)(a[i])           % ค่าน้อยสุด
max(i in 1..n)(a[i])           % ค่ามากที่สุด
forall(i in 1..n)(constraint_i) % AND ทุกตัว
exists(i in 1..n)(constraint_i) % OR อย่างน้อยหนึ่ง
```

Quick Reference: Enum Types

```
% — IN MODEL (.mzn) —————  
enum COLOR = { Red, Green, Blue };    % กำหนดค่าใน model  
enum MACHINES;                        % รับค่าจาก data file
```

```
% — IN DATA FILE (.dzn) —————  
MACHINES = { Lathe, Mill, Drill };
```

```
% — USING ENUM —————  
array[MACHINES] of int: cap = [80, 60, 100];  
var MACHINES: assigned;                % ตัวแปรเป็น enum  
constraint assigned = Lathe;           % กำหนดค่า enum  
constraint forall(m in MACHINES)(load[m] <= cap[m]);
```

```
% — ENUM OPERATIONS —————  
card(MACHINES)    % จำนวนสมาชิก  
min(MACHINES)     % สมาชิกแรก  
max(MACHINES)     % สมาชิกสุดท้าย
```

ข้อผิดพลาดที่พบบ่อย

✗ `array[1..3] of int: x = [1,2,3,4];`

✓ `array[1..4] of int: x = [1,2,3,4];`

ขนาด *index set* ต้องตรงกับ *literal*

✗ `x[0]` % ถ้า *index set* เป็น `1..n`

✓ `x[1]` % MiniZinc เริ่มที่ 1 (ไม่ใช่ 0)

Default *array literal* เริ่ม *index* ที่ 1

✗ `enum E; E = { A, B }; % ใน .mzn`

✓ `% ใน .dzn: E = { A, B };`

ถ้าประกาศ *enum* ไว้ใน *.mzn* ให้ใส่ค่าใน *.dzn*

✗ `if x > 0 then y = 1;`

✓ `if x > 0 then y = 1 else y = 0 endif`

if-then-else ต้องมี *else* และ *endif* เสมอ

สรุปวันที่ 2 — สิ่งที่ต้องจำ



Arrays — จัดการตัวแปรหลายตัว

array[index] of type — เข้าถึงด้วย a[i] — 2D ด้วย a[i,j]



forall + sum — building blocks หลัก

forall = AND ทุกตัว | sum = รวม | exists = OR | where = กรอง



Enum Types — ชื่อแทนตัวเลข

ประกาศ enum PRODUCTS; ใน .mzn → กำหนดค่าใน .dzn → array[PRODUCTS]



if-then-else + bool2int

conditional constraint | bool2int(expr) = 1/0 สำหรับนับเหตุการณ์